SECURITY PROJECT · DETECTION ENGINEERING

Building a Purple-Team Detection Lab with Wazuh

From a bare SIEM install to a working attack → detect loop, mapped to MITRE ATT&CK.

Author: <YOUR NAME> · Background: Security Engineer / Red Team / Bug Bounty · Platform: Wazuh 4.14 on an isolated VMware lab

1 Objectives

I set out to go beyond "install a SIEM and look at the dashboard" and instead practice the actual craft of a **detection engineer**, using my offensive background as the engine:

- Stand up the open-source **Wazuh** XDR/SIEM platform and connect multiple endpoints.
- Run a **purple-team loop**: launch an attack, observe what the SIEM detects (and misses), then **write custom detection rules** to close the gaps.
- Map every detection to **MITRE ATT&CK** and understand the compliance context Wazuh adds.
- Produce something portfolio-worthy: documented, reproducible, and shareable.

2 Architecture

The whole lab runs in VMware on an isolated network so the manager and agents can reach each other while staying off the real network.

- **Kali Linux** — Wazuh **manager**, **indexer**, and **dashboard** (consolidated for the lab), plus attacker tooling (`hydra`, `curl`).
- **Ubuntu Server** — Wazuh **agent**, plus **Apache + MariaDB + PHP + DVWA** as the vulnerable web target.
- **Windows** — Wazuh **agent** for endpoint telemetry.

> **Core mental model:** *the agent collects, the manager decides.* Agents ship raw logs; the manager runs the decoders and rules that actually produce alerts. That single idea explains most of how Wazuh behaves — and caused one of my most instructive mistakes (Section 4).

3 What I built, step by step

- **Connected a Linux agent** to the manager and confirmed it reporting as *Active*.
- **Got a detection "for free":** an SSH brute force (via `hydra`) was caught by Wazuh's built-in rules — failed logins (rule 5710) correlated into a single **brute-force** verdict (rule **5712**, level 10), mapped to **ATT&CK T1110**.
- **Added a second attack class:** stood up **DVWA** and enabled Apache access-log collection on the agent (Wazuh does **not** watch the web log by default). SQL injection and path-traversal payloads were parsed by the `web-accesslog` decoder and flagged by built-in **rule 31104** ("Common web attack", level 6), auto-tagged to ATT&CK and PCI DSS.

- **Authored my first custom rule.** The built-in alert for a `/etc/passwd` traversal was only a generic level-6 note. I wrote **rule 100100 (level 12)** that escalates this specific attack into a high-severity, **attacker-attributed** detection (source IP embedded in the alert) and tagged it to **ATT&CK T1083 — File and Directory Discovery**.

✎ **Add your own results here:** insert screenshots of the Events view showing rule 5712, rule 31104, and your custom rule 100100 firing at level 12, plus any measured counts you confirm from your own run.

4 Challenges & how I solved them

Challenge	Root cause	Fix
Agent couldn't reach the manager ("No route to host")	Manager and agent on **different VMware network modes / subnets**	Put both VMs on the same NAT segment; confirmed reachability with `ping` first
Couldn't copy/paste into the Ubuntu **Server** console	A headless server has no GUI clipboard integration	Enabled SSH and worked from the Kali terminal — paste works natively over SSH
Custom rule produced **no results**	Rule was written **on the agent**; rules only run on the **manager**	Moved `local_rules.xml` to the manager's `/var/ossec/etc/rules/`
Manager threw `XMLERR: elements were not closed`	Malformed / duplicated XML tags in the rules file	Rebuilt the file with balanced tags and re-validated
Rule still didn't appear after the fix	Manager loads rules only at startup; old events don't retroactively re-match	Restarted the manager, then fired a **fresh** attack

> **The habit that tied it together:** validating with `wazuh-logtest` *before* restarting. It parses a sample log line and shows exactly which rule matches and at what level, so a syntax error is caught before it can take the SIEM offline — a genuine production discipline, not just a lab trick.

5 Key takeaways

- **Detection engineering is a loop, not a setting.** Attack → observe → read the raw event → write/tune a rule → validate → verify. Repeating that loop is the skill.
- **Telemetry quality decides everything.** Wazuh caught SSH brute force out of the box but saw nothing on the web until I explicitly added the Apache log. You can't detect what you don't collect.
- **Read the event, don't just count it.** Understanding the decoder, the extracted fields (`srcip`, `url`), the severity model, and the ATT&CK/compliance tags is what lets you write a rule that's *specific* rather than noisy.
- **Specific beats loud.** Going from a generic level-6 alert to a named level-12 detection with the attacker's IP is the difference between noise and an actionable signal.
- **An offensive background is an asset on defense.** Knowing how the attack really works made the detection obvious to design.

6 Next steps

- More custom Linux rules: web-shell upload, directory scanning (repeated 404s), reverse-shell process spawn.
- Install **Sysmon** on the Windows agent and author a Windows detection (e.g. suspicious PowerShell download).
- Build a **MITRE ATT&CK Navigator** heatmap of covered techniques (T1110, T1190, T1083, …).
- Add **active response** to auto-block an attacker IP on a high-severity rule.
- Explore enrichment: VirusTotal on file hashes, the vulnerability-detection module.
- Extend to an **Active Directory** scenario for AD-specific attack detection.

Lab note: this project uses intentionally vulnerable software (DVWA) and was run entirely on an isolated virtual network against machines the author owns.